CS61C

Great Ideas
in
Computer Architecture
(a.k.a. Machine Structures)

Assistant
Teaching Professor
Lisa Yan

SDS IV/V: FSM, Blocks, ALU

Agenda

Accumulator Circuit

- Accumulator Circuit
- FSM
- MUX: Data Multiplexor
- Arithmetic Logic Unit (ALU)

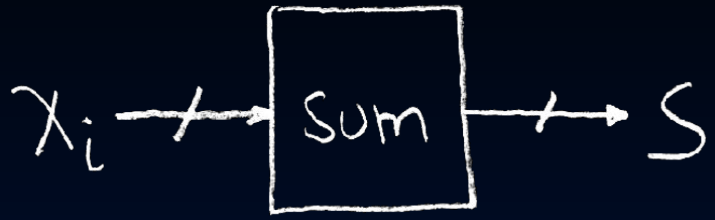
—

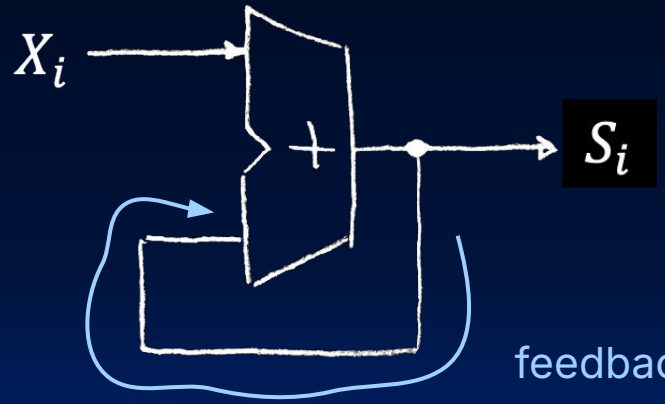
- 1-Bit Adder
- N-Bit Adder with Overflow
- N-Bit Adder and Subtractor

Accumulator Example

- One more motivation for registers:

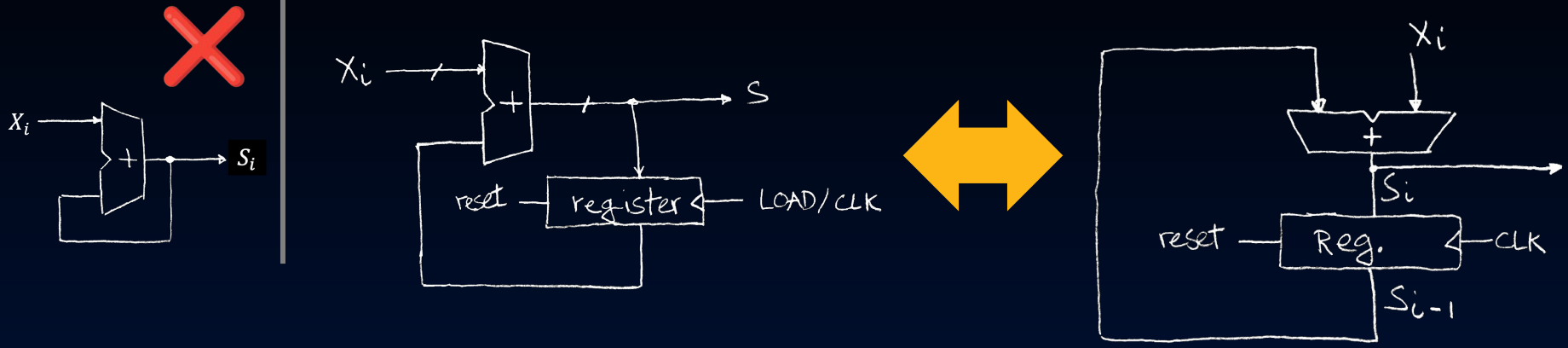
```
S_i = 0; // S_0
for (int i = 0; i < n; i++) {
  S_i = S_i + X_i;
}
```





- Does this work?
- ❌ **No!!!** ❌
  - We add Xi multiple times each clock cycle. Out of control!!

Use a Register to "Pause." One Output per Cycle

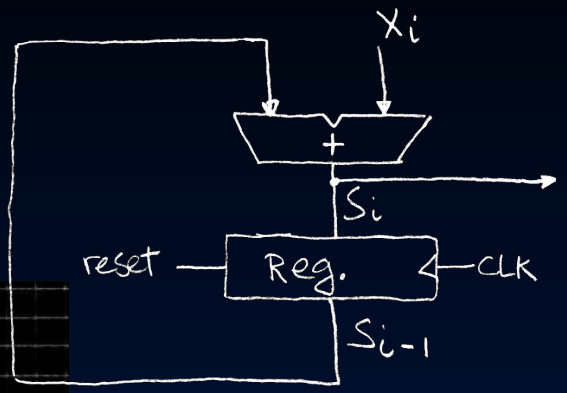


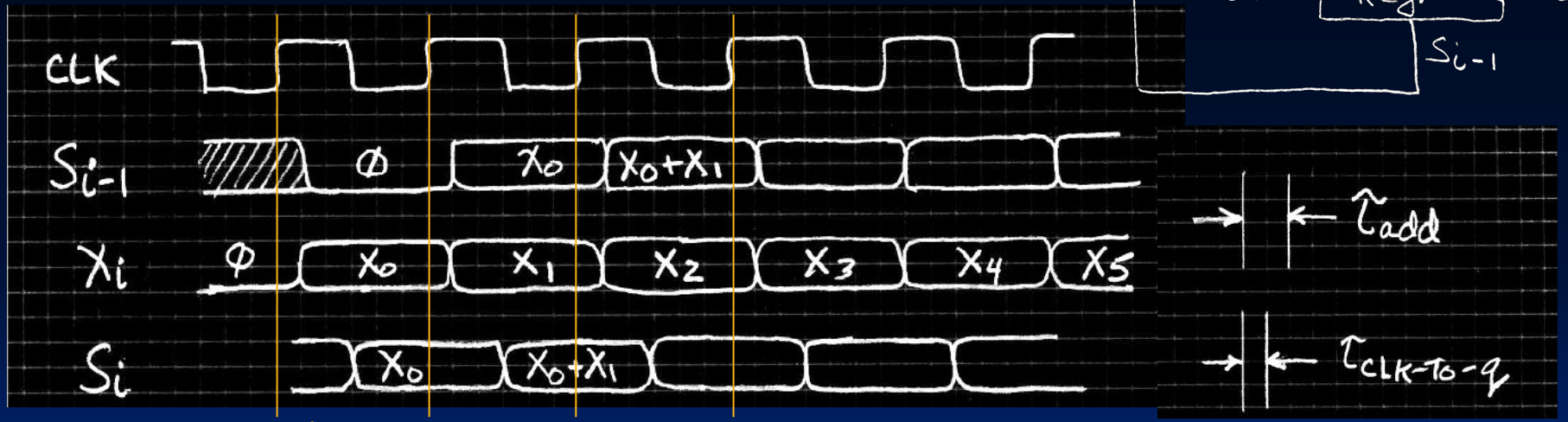
- In the i-th iteration:
  - S_i is the input to the register and the result of the i^{th} iteration.
  - S_{i-1} is the register output and the result of the $(i-1)^{th}$ iteration.

i	0	1	2
S_{i-1}	0	**6**	**8**
X_i	6	2	3
S_i	**6**	**8**	11

Accumulator Waveform Diagram

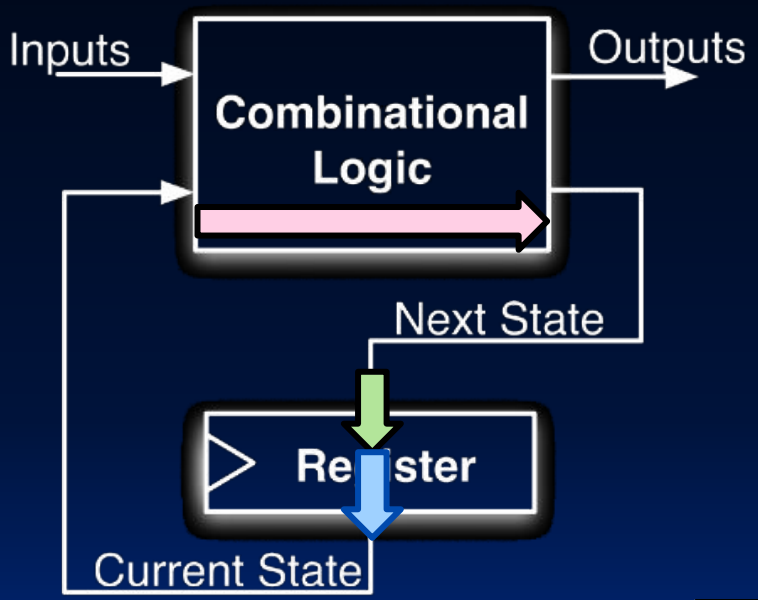
The below waveform diagram shows we can speed up clock frequency!





Critical Path Determines Maximum Clock Frequency

- The minimum clock period is the delay on the **critical path**.
  - **Critical path:** The path between input(s) and output(s) that incurs worst-case (maximum) delay.
- Run the clock any faster, and your circuit becomes unstable!



Max Delay of this Circuit =
Delay of Critical Path =
CLK-to-Q Delay
+ CL Delay
+ Setup Time

Summary: Timing Constraints

- A circuit that runs too fast can cause unstable outputs.

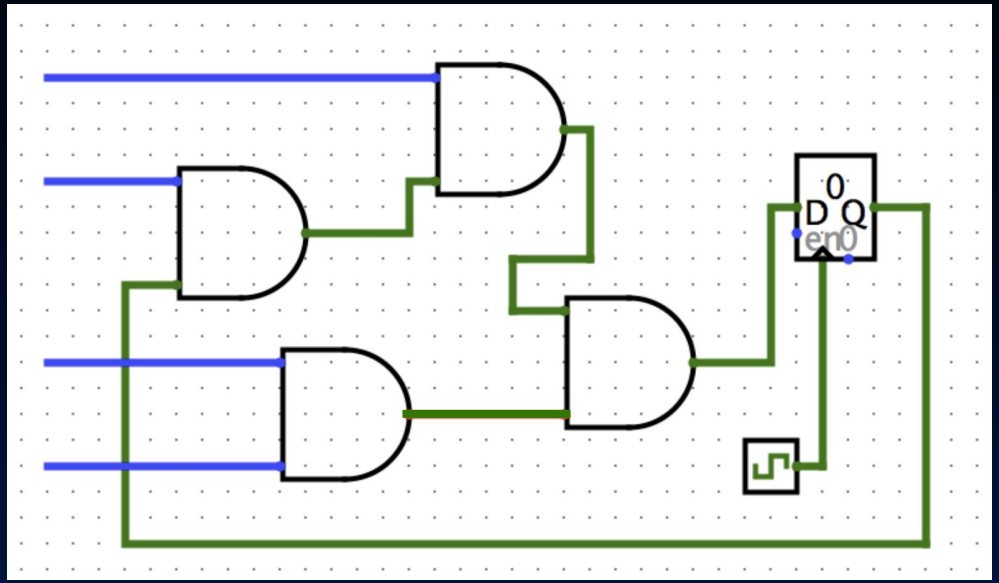
1. Set clock period to be (at minimum) the delay through the critical path.
   - **Critical path**: path between input/output that incurs max delay
   - Set maximum clock frequency = 1/(minimum clock period)

2. [at home] Read about **hold time violations**
   - **Hold time violation**: fastest path between input/output is faster than register hold time (at input wire)

Practice

What is the maximum clock frequency for this circuit?

A. 5 GHz
B. 200 MHz
C. 500 MHz
D. 1/7 GHz
E. 1/6 GHz
F. Something else



clk-to-q delay: 1ns
setup time: 1ns
hold time: 1ns
AND delay: 1ns
(Assume all unconnected inputs come from some register.)

(pause)

How to build functions in hardware?

Agenda

Finite State Machines

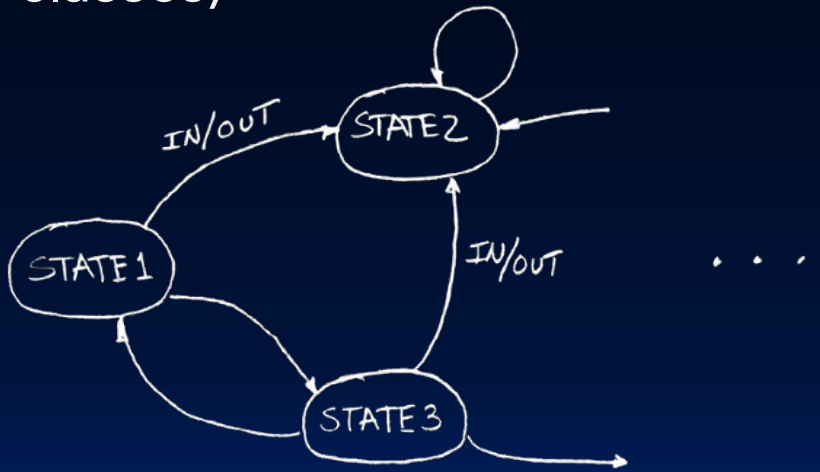
- Accumulator Circuit
- FSM
- MUX: Data Multiplexor
- Arithmetic Logic Unit (ALU)

—

- 1-Bit Adder
- N-Bit Adder with Overflow
- N-Bit Adder and Subtractor

How to build functions in hardware?

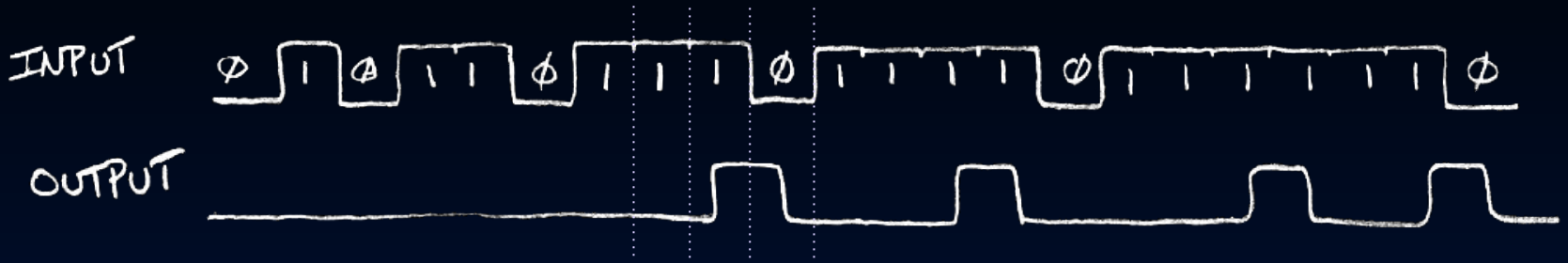
- **Finite State Machine (FSM)**
  - A function that can be represented with a **"state transition diagram."**
  - Terminology on diagram
- (You may have seen FSMs in other classes)



With combinational logic and registers, any FSM can be implemented in hardware.

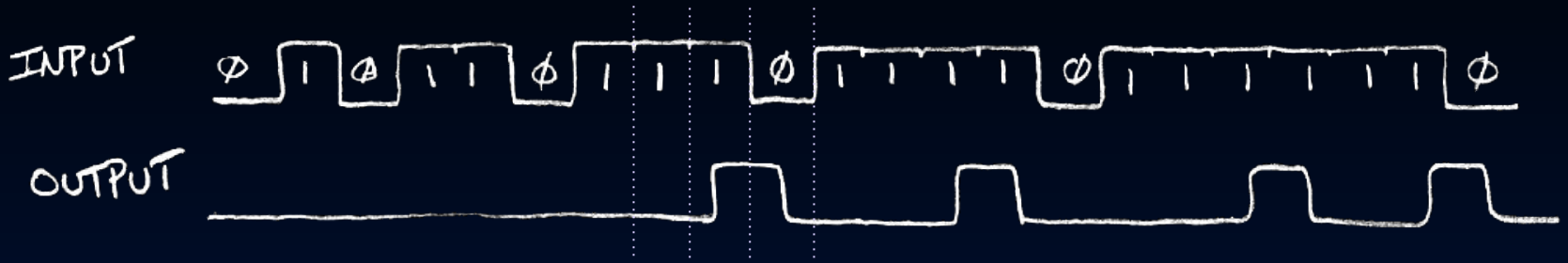
3 Ones: Waveform Diagram

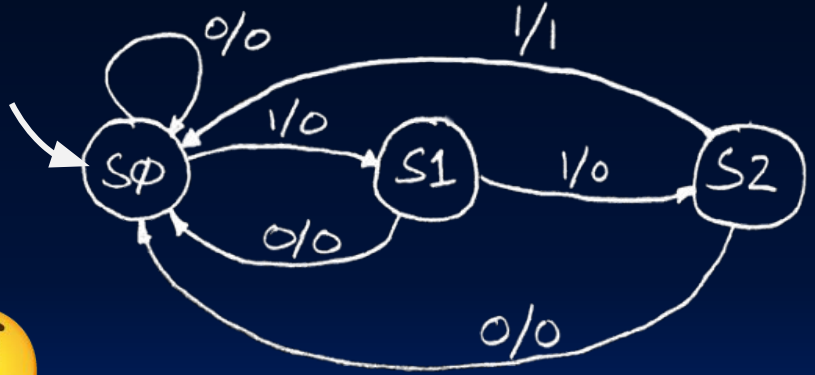
- Function: Detect the occurrence of 3 consecutive 1's in the input.



3 Ones: FSM

- Function: Detect the occurrence of 3 consecutive 1's in the input.





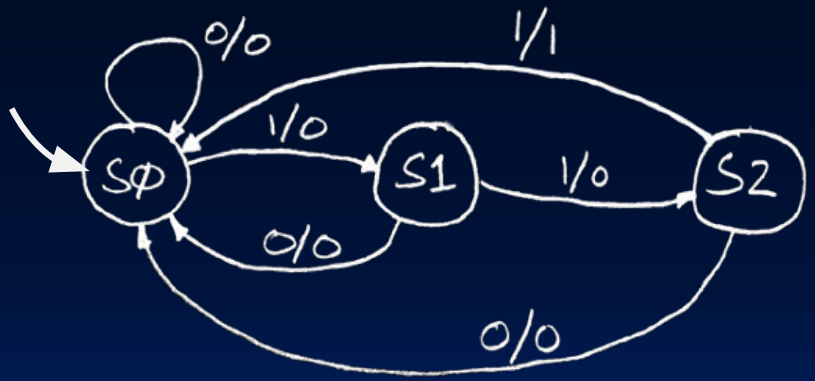
Assume state transitions are controlled by the clock.

How does the FSM (right) produce the waveform diagram (above)?

Hardware Implementation of FSM (1/2)

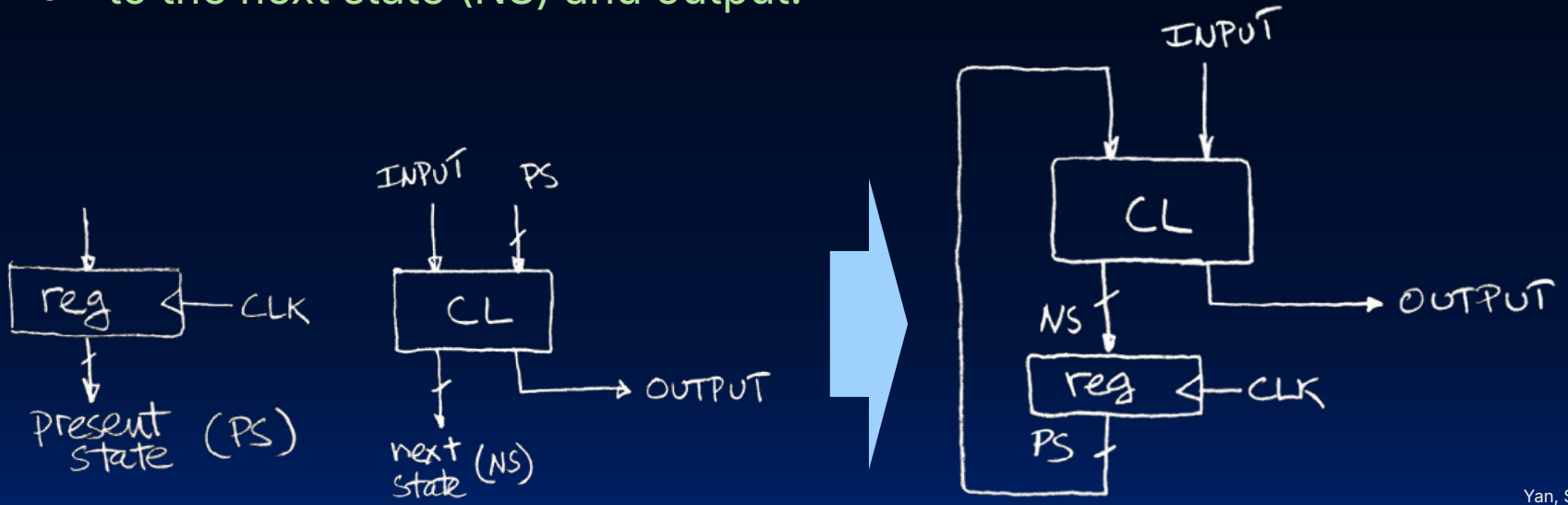
- Assume state transitions are controlled by the clock.
  - On each clock cycle the machine checks the inputs, then
  - It moves to a new state and produces a new output
- Each row of the **truth table** is one edge/transition.

PS	input	NS	output
00	0	00	0
00	1	01	0
01	0	00	0
01	1	10	0
10	0	00	0
10	1	00	1



Hardware Implementation of FSM (2/2)

- A **register** is needed to remember which state the machine is in.
  - Use a **unique bit pattern** for each state.
- **Combinational logic circuit** implements a function that maps:
  - the input and present state (PS)
  - to the next state (NS) and output.



Agenda

MUX: Data Multiplexor

- Accumulator Circuit
- FSM
- MUX: Data Multiplexor
- Arithmetic Logic Unit (ALU)

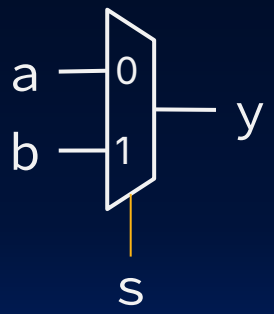
—

- 1-Bit Adder
- N-Bit Adder with Overflow
- N-Bit Adder and Subtractor

A New Block: Multiplexor

- A **multiplexor (MUX)** is a basic logic function whose output is one of the inputs signals selected by a control signal.
  - Also called MUX; **Selector**
  - Implemented with logic gates.

- Example 1: 1-bit wide, 2-to-1 MUX



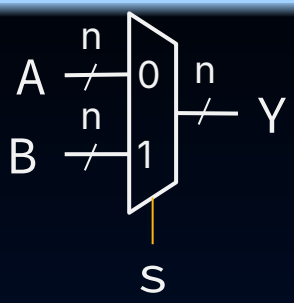
s	a	b	y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

At home: Use sum-of-products to show MUX boolean expression

$$y = \bar{s}a + sb$$

MUX Terminology

- Example 2: n-bit wide, 2-to-1 MUX
  - Data input A, B, output Y all n-bit wide
  - Control s: still 1-bit wide

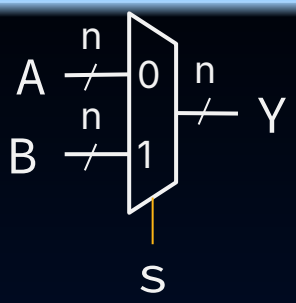


- Example 3: A 32-bit wide 4-to-1 MUX
  - Selects between `__(1)__` input signals, each of which is `__(2)__` bits wide
  - Has `__(3)__` selector bits

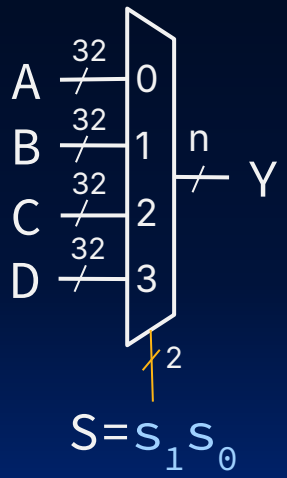
A. 2
B. 4
C. 8
D. 16
E. 32
F. 64
G. 128
H. Something else

MUX Terminology

- Example 2: n-bit wide, 2-to-1 MUX
  - Data input A, B, output Y all n-bit wide
  - Control s: still 1-bit wide



- Example 3: A 32-bit wide 4-to-1 MUX
  - Selects between **4** input signals, each of which is **32** bits wide
  - Has 2 selector bits



S	s_1	s_0	Y
0	0	0	A
1	0	1	B
2	1	0	C
3	1	1	D

Agenda

Arithmetic Logic Unit (ALU)

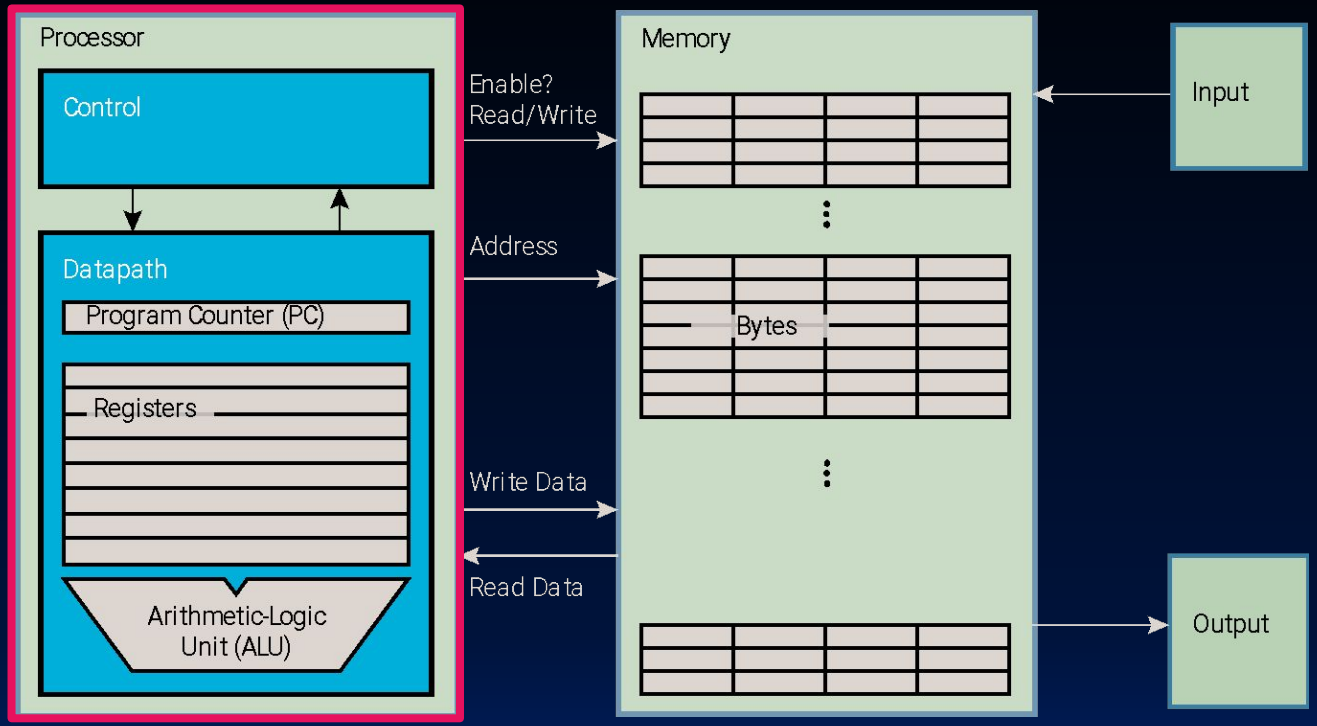
- Accumulator Circuit
- FSM
- MUX: Data Multiplexor
- Arithmetic Logic Unit (ALU)

—

- 1-Bit Adder
- N-Bit Adder with Overflow
- N-Bit Adder and Subtractor

How do we build a Single-Core Processor?

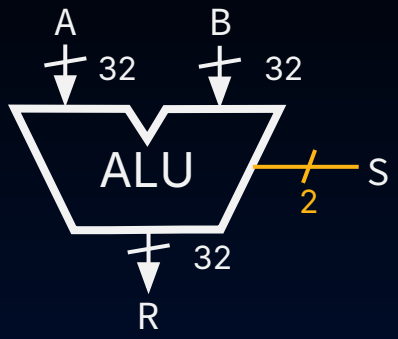
Most Processors contain a special logic block called the **ALU (Arithmetic Logic Unit)**.
The ALU performs arithmetic and bitwise operations.



Today, we will build a **simple 32-bit ALU** that supports 4 operations: AND, OR, ADD, SUBTRACT.

A Simple ALU

- Consider a Simple ALU:
  - Three inputs: 32-bit A, 32-bit B, and 2-bit input **S**
  - **S selects the operation** to perform on A and B.
  - Outputs 32-bit **R**, the result of performing the selected operation on A and B.

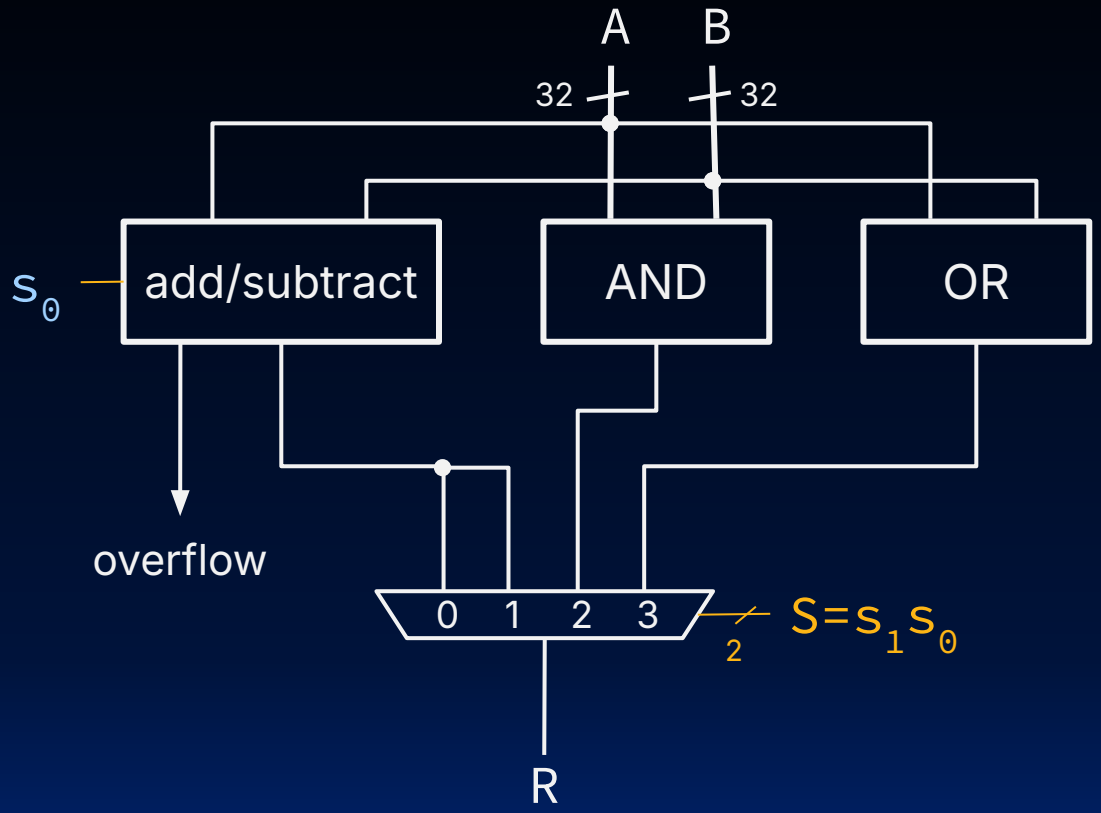


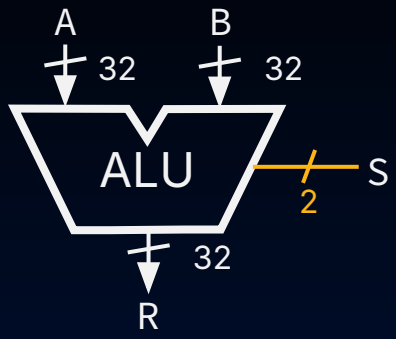
- Suppose we consider 4 operations:
  - ADD
  - SUBTRACT
  - AND
  - OR

S		
S_1	S_0	R
0	0	A+B
0	1	A−B
1	0	A&B
1	1	A|B

A Simple ALU

In this circuit, **all results are computed** but exactly one is selected.





S		
s_1	s_0	R
0	0	A+B
0	1	A−B
1	0	A&B
1	1	A|B

Agenda

FOR NEXT TIME

1-Bit Adder

- Accumulator Circuit
- FSM
- MUX: Data Multiplexor
- Arithmetic Logic Unit (ALU)

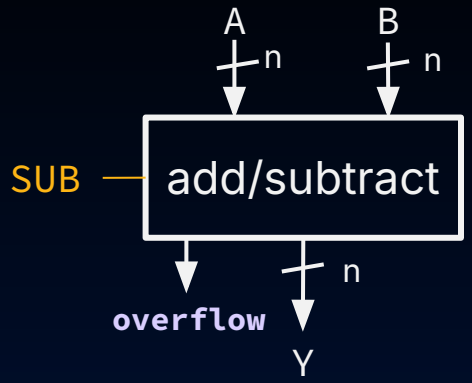
—

- 1-Bit Adder
- N-Bit Adder with Overflow
- N-Bit Adder and Subtractor

Adder/Subtractor Design

Perform arithmetic with one block:

- SUB input bit: 1 if subtraction, 0 if addition
- **Overflow**: 1 if signed arithmetic overflow occurred



For now, we focus on **addition**.

- Explore a design for an **n-bit adder**, where n = 32, 4, etc.

N-Bit Adder = Cascading Intuition

- For now, let's focus on **addition**.
  - Consider adding two 4-bit numbers:
    - $A=a_3a_2a_1a_0$
    - $B=b_3b_2b_1b_0$

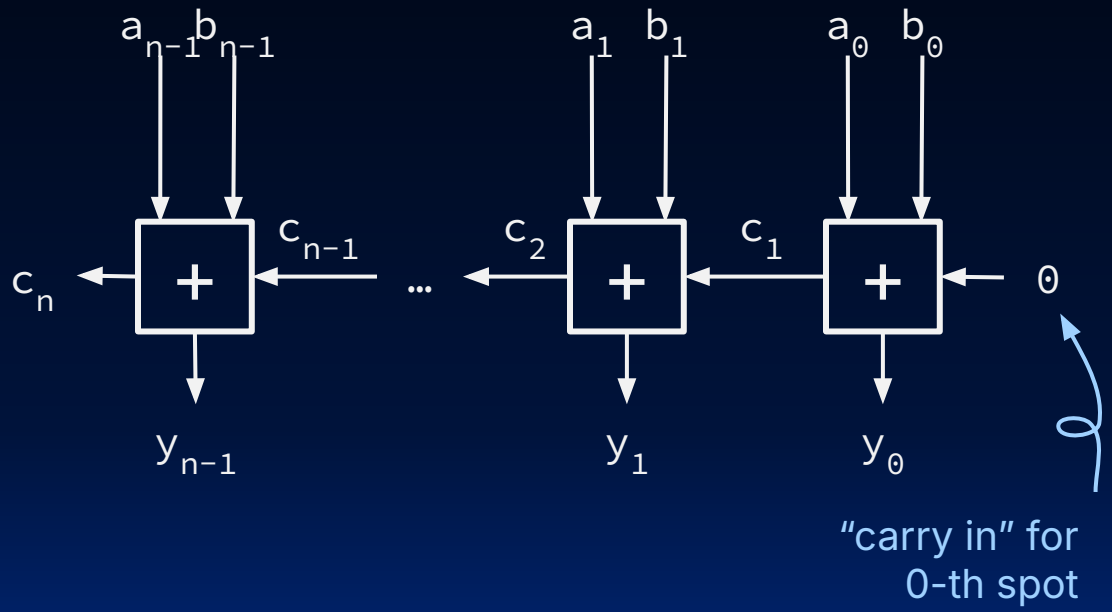
$$\begin{array}{rcccc} & c_3 & c_2 & c_1 & \\ & a_3 & a_2 & a_1 & a_0 \\ + & b_3 & b_2 & b_1 & b_0 \\ \hline & y_3 & y_2 & y_1 & y_0 \end{array}$$

- Addition in the i-th spot produces **two outputs**:
  - **result** (e.g., y_i)
  - **carry bit** (e.g., c_{i+1}) which feeds into the (i+1)-th spot addition.

```
  0110 carry bits
  1011
+ 0001
------
  1100
```

N-Bit Adder = Cascading 1-Bit Adders

$$
\begin{array}{rcccc}
 & c_3 & c_2 & c_1 & \\
 & a_3 & a_2 & a_1 & a_0 \\
+ & b_3 & b_2 & b_1 & b_0 \\
\hline
 & y_3 & y_2 & y_1 & y_0
\end{array}
$$

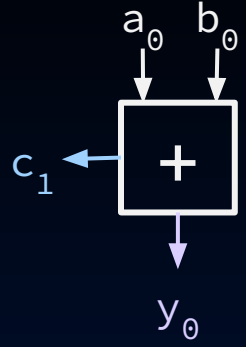


Instead of computing canonical form (e.g. via Sum Product), deconstruct our n-bit adder into a **cascade** of 1-bit adders.

(1/4) One-bit Adder: 0-th Spot (without Carry In Bit)

a_0	b_0	y_0	c_1
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

$$
\begin{array}{cccc|c}
 & c_3 & c_2 & c_1 & \\
 & a_3 & a_2 & a_1 & a_0 \\
+ & b_3 & b_2 & b_1 & b_0 \\
\hline
 & y_3 & y_2 & y_1 & y_0
\end{array}
$$



$y_0 = \text{XOR}(a_0, b_0)$

$c_1 = \text{AND}(a_0, b_0)$

$= a_0 b_0$

XOR: 1 only if the #s of 1s at its input is odd.

$$
\begin{array}{rcl}
 & 11 & \text{carry bits} \\
 & 1011 & \\
+ & 0001 & \\
\hline
 & 1100 &
\end{array}
$$

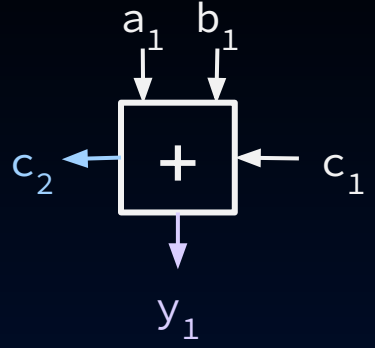
(2/4) One-bit Adder: 1-th Spot (with Carry In Bit)

a_1	b_1	c_1	y_1	c_2
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

y_1 = _____

c_2 = _____

$$\begin{array}{cccc} c_3 & c_2 & \boxed{c_1} & \\ a_3 & a_2 & a_1 & a_0 \\ + \; b_3 & b_2 & b_1 & b_0 \\ \hline y_3 & y_2 & y_1 & y_0 \end{array}$$



A. XOR(a_1, b_1)

B. AND(a_1, b_1)

C. XOR(a_1, b_1, c_1)

D. AND(a_1, b_1, c_1)

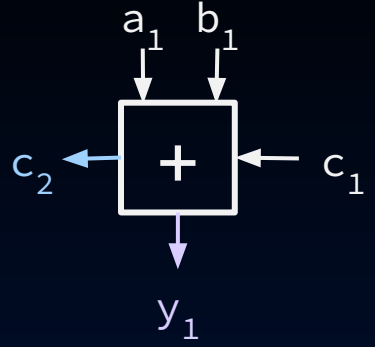
E. MAJ(a_1, b_1, c_1) (Majority bit)

F. Something else

(3/4) One-bit Adder: 1-th Spot (with Carry In Bit)

a_1	b_1	c_1	y_1	c_2
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$
\begin{array}{cccc}
 & c_3 & c_2 & \boxed{c_1} \\
 & a_3 & a_2 & \boxed{a_1}\ a_0 \\
+ & b_3 & b_2 & \boxed{b_1}\ b_0 \\
\hline
 & y_3 & y_2 & \boxed{y_1}\ y_0
\end{array}
$$



$y_1 = \mathbf{XOR}(a_1, b_1, c_1)$

$c_2 = \mathbf{MAJ}(a_1, b_1, c_1)$

$= a_1b_1 + b_1c_1 + a_1c_1$

XOR: 1 only if the #s of 1s at its input is odd.

MAJ ("majority"): see earlier

See notes for gate diagram

Agenda

N-Bit Adder with Overflow

- Accumulator Circuit
- FSM
- MUX: Data Multiplexor
- Arithmetic Logic Unit (ALU)

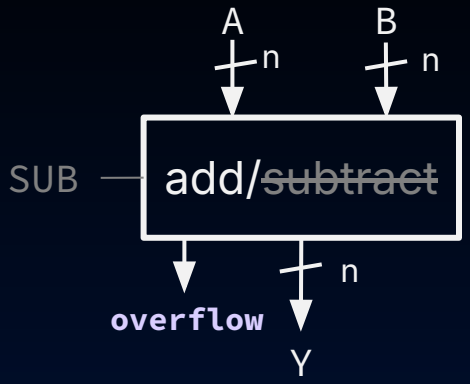
—

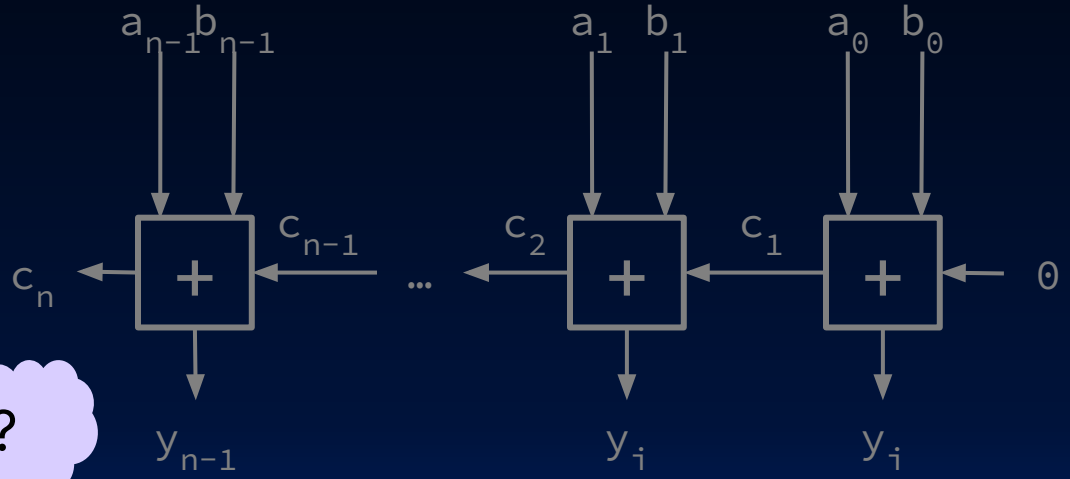
- 1-Bit Adder
- N-Bit Adder with Overflow
- N-Bit Adder and Subtractor

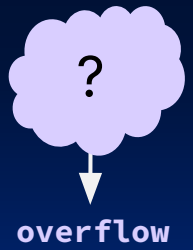
Adder: Determining Overflow

Perform arithmetic with one block:

- SUB input bit: 1 if subtraction, 0 if addition
- **Overflow**: 1 if signed arithmetic overflow occurred

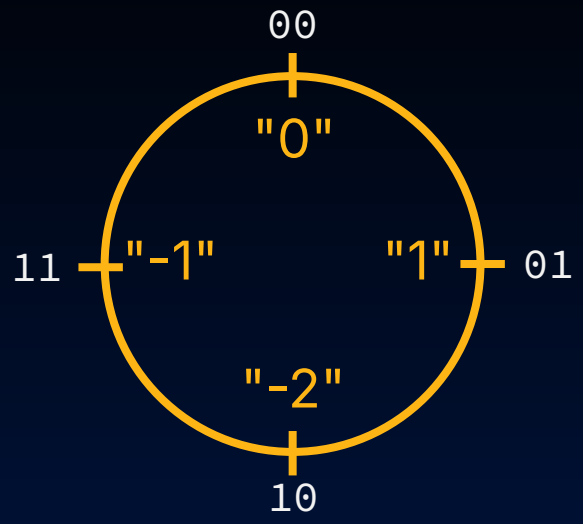






How to compute overflow bit assuming that A, B are signed integers?

2-bit Signed Addition: Overflow (1/4)



A \ B	00 "0"	01 "1"	10 "-2"	11 "-1"
11 "-1"	"0"	"1"	"-2"	"-1"
10 "-2"	"-2"	"-1"	"0"	"1"
01 "1"	"-2"	"-1"	"0"	"1"
00 "0"	"0"	"1"	"-2"	"-1"

Boxed inputs incur **signed overflow**.

2-bit Add: Visualize with Carry Bits (2/4)

Define "last stage" carry bits:

- c_{n-1} : carry in (e.g., c_1)
- c_n : carry out (e.g., c_2)

What is their relationship when overflow occurs? Doesn't occur?

A. AND(c_n, c_{n-1})
B. OR(c_n, c_{n-1})
C. NAND(c_n, c_{n-1})
D. NOR(c_n, c_{n-1})
E. XOR(c_n, c_{n-1})
F. Something else

+	00 "0"	01 "1"	10 "-2"	11 "-1"
11 "-1"	11 +11 "-2" 110	1 11 +11 "-2" 110	1 11 +11 "-2" 110	1 11 +11 "-2" 110
10 "-2"	10 +10 "0" 100	10 +10 "0" 100	10 +10 "0" 100	10 +11 "1" 101
01 "1"	01 +01 "-2" 010	01 +01 "-2" 010	01 +10 "-1" 011	01 +11 "0" 100
00 "0"	00 +00 "0" 000	00 +01 "1" 001	00 +10 "-2" 010	00 +11 "-1" 011

2-bit Signed Addition: Overflow (4/4)

Define "last stage" carry bits:

- c_{n-1} : carry in (e.g., c_1)
- c_n : carry out (e.g., c_2)

Relationship:

- Without overflow: $c_{n-1} = c_n$
- With overflow: $c_{n-1} \neq c_n$

overflow = XOR(c_n, c_{n-1})

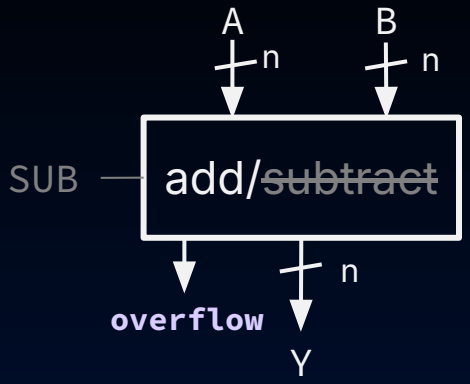
Convince yourself this works for adding any n-bit signed integers!

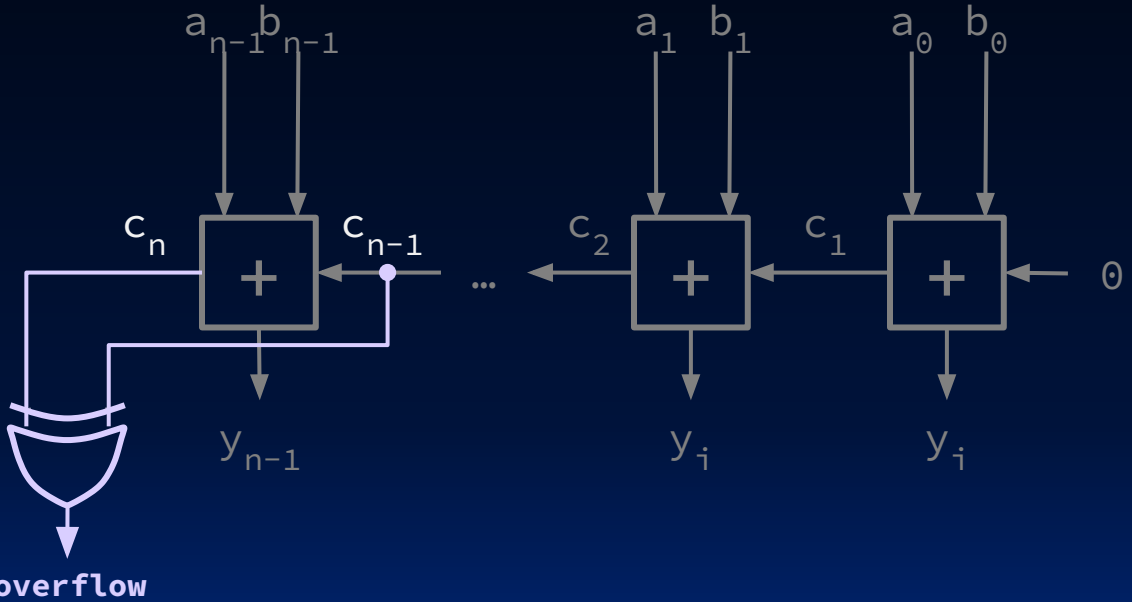
+	00 "0"	01 "1"	10 "-2"	11 "-1"
00 "0"	0 00 +00 "0" 000	0 00 +01 "1" 001	0 00 +10 "-2" 010	0 00 +11 "-1" 011
01 "1"	0 01 +00 "1" 001	1 01 +01 "-2" 010	0 01 +10 "-1" 011	1 01 +11 "0" 100
10 "-2"	0 10 +00 "-2" 010	0 10 +01 "-1" 011	0 10 +10 "0" 100	0 10 +11 "1" 101
11 "-1"	0 11 +00 "-1" 011	1 11 +01 "0" 100	1 11 +10 "1" 101	1 11 +11 "-2" 110

Adder: Overflow is XOR of Carry Bits

Perform arithmetic with one block:

- SUB input bit: 1 if subtraction, 0 if addition
- **Overflow**: 1 if signed arithmetic overflow occurred





Agenda

N-Bit Adder and Subtractor

- Accumulator Circuit
- FSM
- MUX: Data Multiplexor
- Arithmetic Logic Unit (ALU)

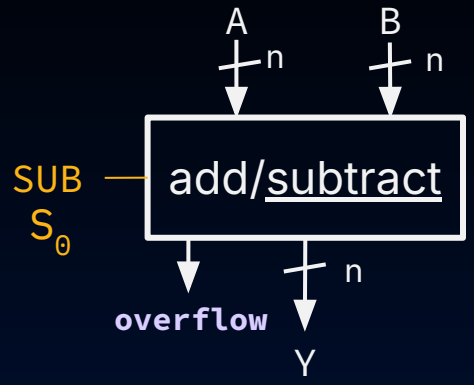
—

- 1-Bit Adder
- N-Bit Adder with Overflow
- N-Bit Adder and Subtractor

~~Adder~~/Subtractor Design

Perform arithmetic with one block:

- SUB input bit: 1 if subtraction, 0 if addition
- **Overflow**: 1 if signed arithmetic overflow occurred



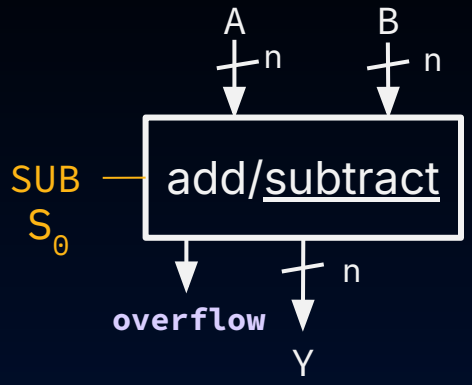
Signed integers with two's complement!

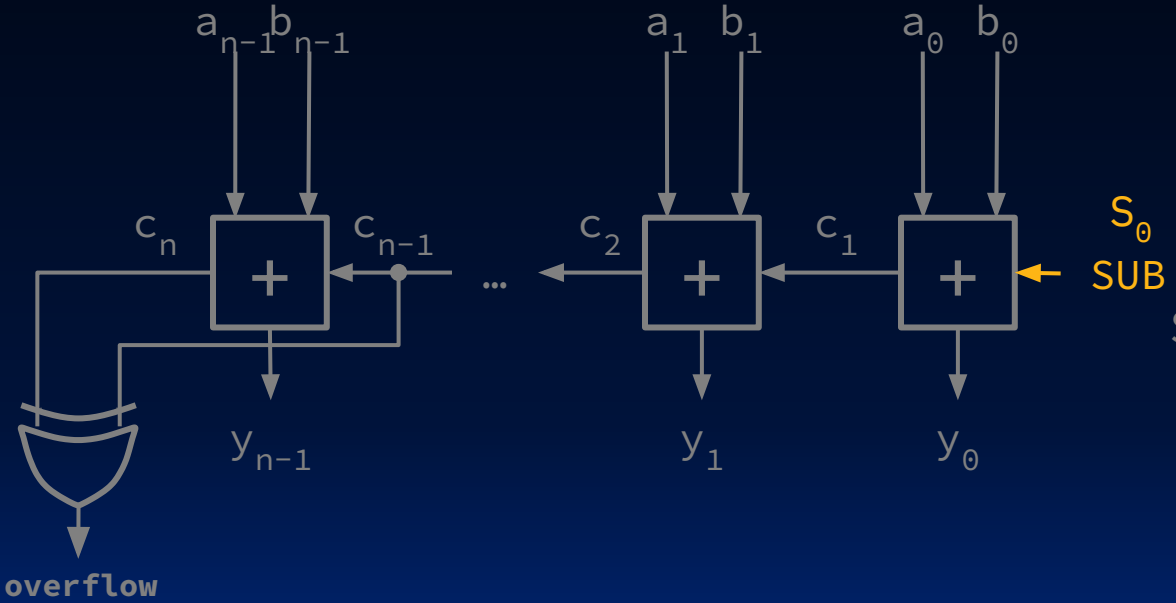
SUB signals add or subtract:

```
0: Y = A + B
1: Y = A + (-B)
     = A + ~B + 1
```

bitwise inversion

Subtractor Design: Extremely Clever! (1/2)





SUB signals add or subtract:

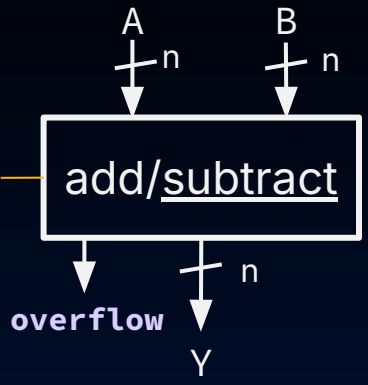
```
0: Y = A + B
1: Y = A + (-B)
     = A + (~B + 1)
```

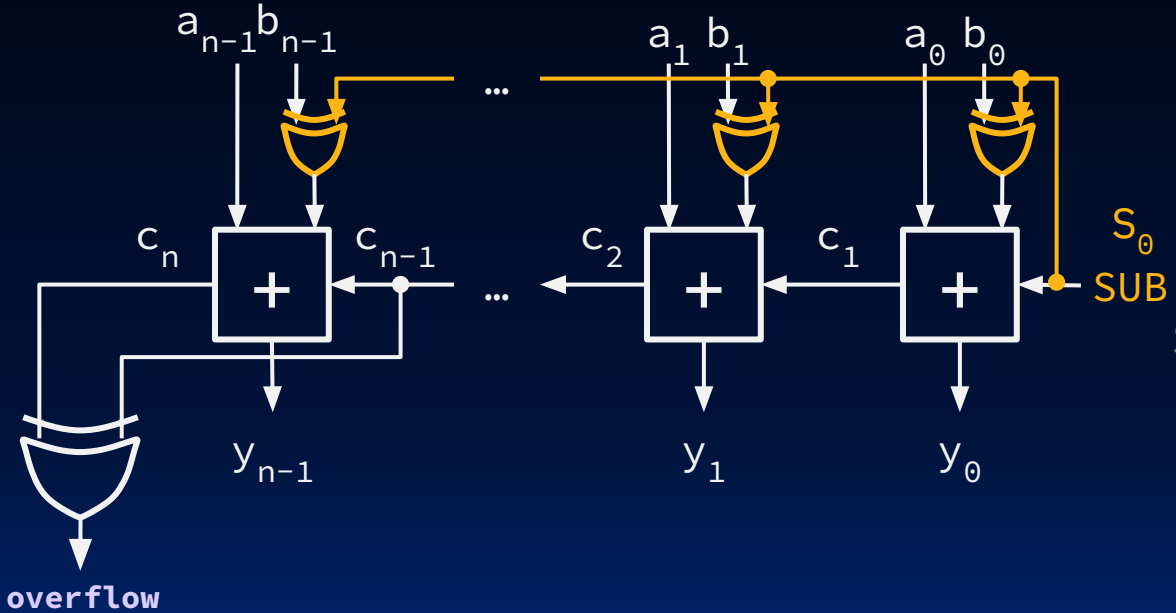
c_0=SUB

Subtractor Design: Extremely Clever! (2/2)

XOR is a conditional inverter! See notes

SUB	XOR(SUB,0)	XOR(SUB,1)	
0	0	1	pass through
1	1	0	invert



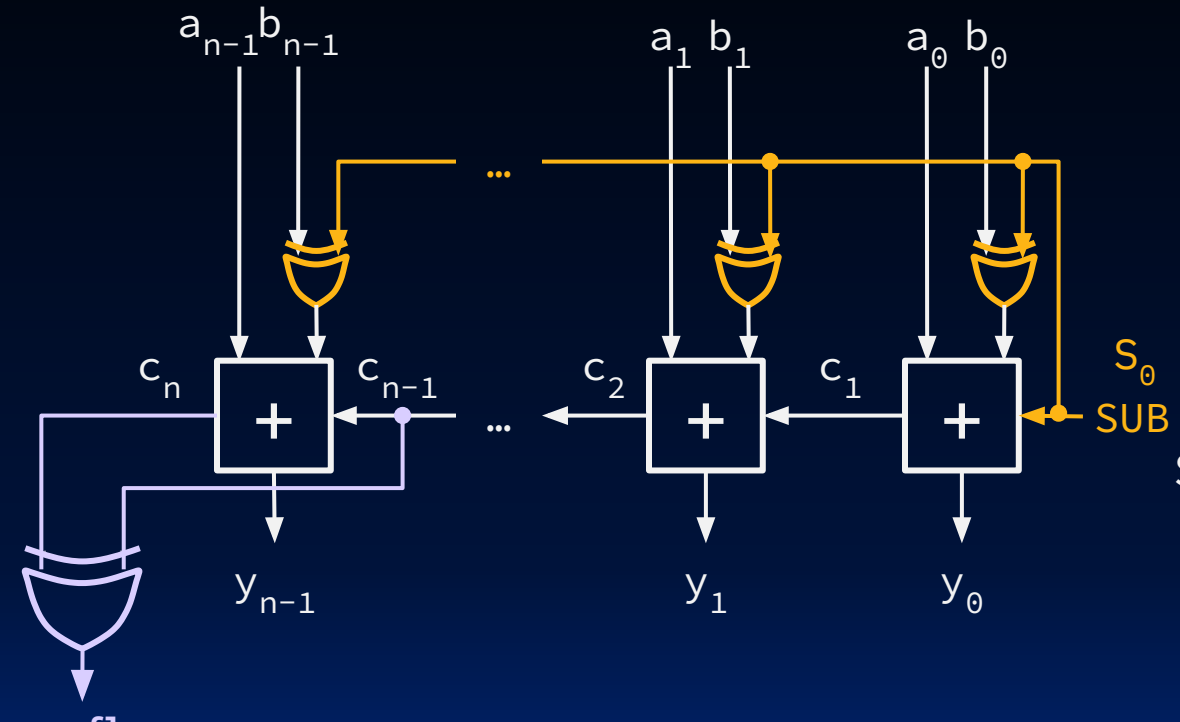


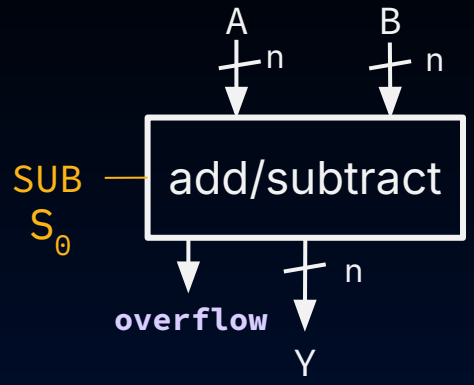
SUB signals add or subtract:

```
0: Y = A + B
1: Y = A + (-B)
     = A + (~B + 1)
```

XOR with SUB

N-Bit Adder/Subtractor, Summary





SUB signals add or subtract:

```
0: Y = A + B
1: Y = A + (-B)
     = A + (~B + 1)
```

"And In conclusion…"

- Use muxes to select among input
  - S input bits selects 2^S inputs
  - Each input can be n-bits wide, indep of S
- Can implement muxes hierarchically
- ALU can be implemented using a mux
  - Coupled with basic block elements
- N-bit adder-subtractor done using N 1-bit adders with XOR gates on input
  - XOR serves as conditional inverter